

PYTHON編程—文字式RPG遊戲製作

RPG遊戲我做主

資料類別的應用

遊戲設計的基本概念及流程

- 遊戲概念的產生
- 遊戲規則的設計
- 遊戲開發的步驟
- 遊戲測試
- 遊戲的除錯及平衡性調整
- 遊戲發佈

你能說一下你喜歡的遊戲有甚麼吸引之處嗎？
能說出有那些元素最吸引你？

RPG遊戲

- 角色扮演遊戲（英語：Role-Playing Game，簡稱RPG）是一種遊戲類型，在遊戲中，玩家扮演虛擬世界中的一個或者幾個角色進行遊戲，玩家通過操控遊戲角色與敵人戰鬥，提升等級、收集裝備和完成遊戲設置的任務，並體驗劇情。通常這類遊戲都是由玩家扮演角色在遊戲世界中漫遊，而一路上的各種遭遇（如戰鬥、交談、會見重要人物等）則是玩家人物成長及遊戲進行的重要關鍵所在。

你能說出幾款RPG遊戲嗎？
他們有哪些特色？



RPG遊戲的組成部分

- 角色/敵人
 - 名字
 - 生命值
 - 屬性
 - 技能
 - 等級
- 戰鬥
 - 回合
 - 實時
- 任務
- 裝備 / 道具
- 故事
- 場景



遊戲概念設計(不插電活動)

- 請兩張枱為一組(約4人)，分組討論，並在討論結束後分享以下內容：
 - 小組提出一個簡單的遊戲
 - 遊戲的類型是甚麼
 - 遊戲的目標、玩法和規則
 - 你們組的遊戲有何特色

RPG角色設計及原則

- 請按以同座為小組的齊來設計一下RPG遊戲角色
 - 角色的名稱
 - 角色的屬性
 - 角色的能力
 - 主角、敵人的特色

關於PRINT的三引號

- 三引號是可以換行寫代碼（有時候代碼過長，換行會便於理解和閱讀），直接打三個單引號或者雙引號（必須一對，不能是開始單引號，結尾是雙引號這種的），換行輸出，在以三引號結尾就可。

- 程式如下：

```
print("""
```

以下是print的特別用法

可以根據輸入情況

同樣地進行輸出

```
""")
```

以下是print的特別用法
可以根據輸入情況
同樣地進行輸出

利用三引號，我們可以
為即將製作的遊戲，設
計出特別的圖案效果。

一隻可怕的怪物出現了！

```
^^|___|^^  
!|-_|!  
|~~~|
```

你要與之戰鬥嗎？

小結

- 遊戲設計的基本概念和角色建立的步驟。
- 使用Python建立角色的字典並設定初始屬性
 - Dictionary字典
 - List串列

```

RPG-dictionary.py
1 import random #導入隨機數庫
2
3 # 定義怪物"列表/串列list"(使用"字典dictionary"對應名稱及相關屬性)
4 monsters = [
5     {"名稱": "IT史萊姆", "生命": 20, "攻擊": 5, "閃避": 0.3},
6     {"名稱": "IT哥布林", "生命": 50, "攻擊": 15, "閃避": 0.1},
7     {"名稱": "IT巨魔", "生命": 100, "攻擊": 20, "閃避": 0.05}
8 ]
9
10 # 玩家初始屬性
11 player_life = 100 #生命值
12 player_attack = 20 #攻擊力
13 player_defense = 0.1 #防禦值
14 player_dodge = 0.15 #閃避值
15
16 # 遊戲開始
17 print("一隻可怕的怪物出現了!")
18 print("^^|_|^^")
19 print("!|-! ")
20 print("|~~| ")
21 print("你要與之戰鬥嗎? ")
22
23 while True: #使用while True進入遊戲的循環中
24     monster = random.choice(monsters) # 從列表/串列list中, 隨機取出一個項目隨機選擇一個值"怪物"
25     # 以下是玩家回合內容
26     print("\n你的回合到了:")
27     print("你的生命值: ", player_life)
28     print("怪物的生命值: ", monster["生命"])

```

```

遊戲環境
一隻可怕的怪物出現了!
^^|_|^^
!|-!
|~~|
你要與之戰鬥嗎?

你的回合到了:
你的生命值: 100
怪物的生命值: 50
1. 攻擊
2. 防禦
3. 逃跑
請選擇你的行動: |

```

資料類別：串列和字典

做個創造者

為了能統
一批量處
理資料，
PYTHON
中加入了
LIST等數
據類型

使用list來表示小僑的四個技能是這樣的：

```
list_skills = ["治療","疾跑","狂暴","干擾"]
```

使用list來表示多個學生的成績是這樣的：

```
list_scores = [98, 89, 78, 97, 96, 55 , 79 , 99, 81, 84, 86]
```

有了索引之
後，結合迴
圈我們就能
對整個LIST
的所有資料
做統一處理

比如將小僑的所有技能輸出顯示
`list_skills = ["治療","疾跑","狂暴","干擾"]`

```
for i in range(4):  
    print(list_skills[i])
```

比如判斷全校所有學生的分數

```
list_scores = [98, 89, 78, 97, 96, 75, 79, 55 , 99, 82, 81, 84, 86]  
for i in range(100):  
    if list_scores[i] >= 60:  
        print( "及格" )  
    else:  
        print("不及格")
```

當然，在
使用索引
時還需要
注意下標
的取值範
圍

留意例子中的range(1000)

```
1 list_scores = [98, 89, 78, 97, 96, 75, 79,
2 for i in range(1000):
3     if list_scores[i] >= 60:
4         print("及格" , end=" ")
5     else:
6         print("不及格", end=" ")
7
```

雖然能輸出結果，報出錯誤：list的下標越界了

```
>>> %Run -c $EDITOR_CONTENT
及格 及格 及格 及格 及格 及格 及格 不及格 及格 及格 及格 及格 及格
Traceback (most recent call last):
  File "<string>", line 3, in <module>
IndexError: list index out of range
... |
```

取出列表的資料

索引

- 相當於書籍中的目錄一樣
- 通過頁碼或者序號
- 可以快速的找到對應的內容

```
list_skills = ["治療","疾跑","狂暴","干擾"]  
print(list_skills[1])  
print(list_skills[4])
```

```
>>> %Run -c $EDITOR_CONTENT  
疾跑  
Traceback (most recent call last):  
  File "<string>", line 3, in <module>  
IndexError: list index out of range  
>>>
```

取值時，如果超出索引範圍就會報錯。

取出列表的資料

切片

- 取出列表中的指定數據

[start : end : step]

[開始 : 結束 : 間隔]

```
list_skills = ["治療", "疾跑", "狂暴", "干擾"]
```

```
print(list_skills[:1:2])
```

```
print(list_skills[::3])
```

```
print(list_skills[1:2])
```

```
print(list_skills[::-1]) #反轉列表
```

```
>>> %Run -c $EDITOR_CONTENT
```

```
['治療']
```

```
['治療', '干擾']
```

```
['疾跑', '干擾']
```

```
['干擾', '狂暴', '疾跑', '治療']
```

LIST的常規操作

insert
插入

```
list_skills = ["治療", "疾跑", "狂暴", "干擾"]  
list_skills.insert(1, "淨化")  
print(list_skills)
```

```
0      1      2      3      4  
['治療', '淨化', '疾跑', '狂暴', '干擾']
```

append
增加

```
list_skills = ["治療", "疾跑", "狂暴", "干擾"]  
list_skills.append("淨化")
```

```
>>> %Run -c $EDITOR_CONTENT  
['治療', '疾跑', '狂暴', '干擾', '淨化']
```

remove
刪除

```
list_skills = ["治療", "疾跑", "狂暴", "干擾", "淨化"]  
list_skills.remove("淨化")  
print(list_skills)
```

```
>>> %Run -c $EDITOR_CONTENT  
['治療', '疾跑', '狂暴', '干擾']
```

串列/列表 的常用操作小結

分類	函數/方法	說明
增加	串列.insert(索引,數據)	在指定位置插入數據,位置前有空元素會補上
	串列.append(數據)	在末尾追加數據
	串列.extend(串列)	讓一個串列的後方，合併另外一個串列的內容。 (會改變串列，但不會回傳串列的值，)
刪除	del 列表[索引]	刪除指定索引的數據
	串列.remove(數據)	刪除第一個指定數據
	串列.pop()	刪除末尾最後一個數據，返回被刪除的元素
	串列.pop(索引)	刪除指定索引數據
	串列.clear	清空整個串列的內容，變成一個空的串列。

串列/列表 的常用操作小結

分類	函數/方法	說明
修改	串列[索引]=數據	修改指定索引的數據。如:list1[1]= "mbc"
查詢	串列[索引]	根據索引取值
	串列.index(數據)	根據值查詢索引,返回首次出現時的索引
	串列.count(數據)	數據在串列中出現的次數
	len(串列)	返回串列的長度
	if數據 in 串列	檢查列表中是否包含某元素
排序	串列.sort()	內容按升序排序1,2,3,4,5,6
	串列.sort(reverse=True)	內容按降序排序6,5,4,3,2,1
	串列.reverse()	將串列反轉

字典 DICTIONARY

- 顧名思義就像「查詢用的字典」一樣，透過要查詢的「鍵 key」(關鍵字)，就能夠查詢到對應的「值 value」(解釋說明)，也是使用頻率相當高的資料型態。
- 字典跟串列類似，都能作為儲存資料的容器，可以放入字串、整數、布林、串列或字典。
- 字典的格式：`{ '鍵' : 值 }`
- 鍵必須是「字串」，前後必須加上「引號」，值則可以是任意的 Python 物件 (字串、數字、串列、字典...等)，不同鍵值中間使用逗號「,」分隔。

字典的用法

- 建立字典

大括號 {}--使用方式為「{
鍵':值}」，鍵必須是「字串」

```
a = {'name':'oxxo','age':18,'eat':['apple','banana']}
```

dict()--使用「dict(鍵=
值)」，鍵的前後「不需要
加上引號」

```
a = dict(name='oxxo', age=18, eat=['apple','banana'])
```

- 讀取字典

中括號 []--使用 b =
a['name']

```
a = {'name':'oxxo', 'age':18, 'eat':['apple','banana']}  
b = a['eat'][0]
```

使用中括號取值時，如果沒有對應的鍵，就會發生錯誤，如果要避免這種情況，
就可以使用 `get()` 函式來取值。

get()--有鍵就會回傳值，
如果找不到鍵，就會回傳
None

```
a = {'name':'oxxo', 'age':18, 'eat':['apple','banana']}  
c = a.get('name')
```

字典的用法

- 修改字典

中括號 []--
a['name'] =
'XXXX'

```
a = {'name': 'XXXX', 'age':18}  
a['name'] = 'gilbert'
```

- 刪除字典

del · clear

```
a = {'name':'oxxo', 'age':18}  
del a['name']
```

```
a = {'name':'oxxo', 'age':18}  
a.clear()
```

齊來使用PYTHON建立角色

一隻可怕的怪物出現了！

```
^^|__|^  
!|- -|!  
|~~~|
```

你要與之戰鬥嗎？

- List(串列)能儲存多個角色或道具，如主角團、敵人或道具庫等

```
1 # 定義怪物團(將"字典dictionary"嵌套入"列表/串列"中)
2 monsters = [
3     {"名稱": "IT史萊姆", "生命": 20, "攻擊": 5, "閃避": 0.3},
4     {"名稱": "IT哥布林", "生命": 50, "攻擊": 15, "閃避": 0.1},
5     {"名稱": "IT巨魔", "生命": 100, "攻擊": 20, "閃避": 0.05}
6 ]
7
8 # 模擬遊戲開始遇怪
9 print("一隻可怕的怪物出現了！")
10 print("^^|__|^")
11 print(" !|- -|!")
12 print(" |~~~| ")
13 print("你要與之戰鬥嗎？")
14
15 #在list中讀取其中一隻怪
16 monster1=monsters[0]
17 print(monster1["名稱"])
```

while True: / while 1:

遊戲啟動1：怪物出現

RANDOM(隨機數)

- Python 的標準函數式「random」提供了產生隨機數的方法，不論是隨機整數、隨機浮點數，或要從串列中隨機取值，都能透過 random 標準函式來實現。本次我們將實現在list串列中隨機取數。

random.choice(list列表)
可以從一個字串、串
列或 tuple 中 (有順序
的物件)，隨機取出
一個項目。

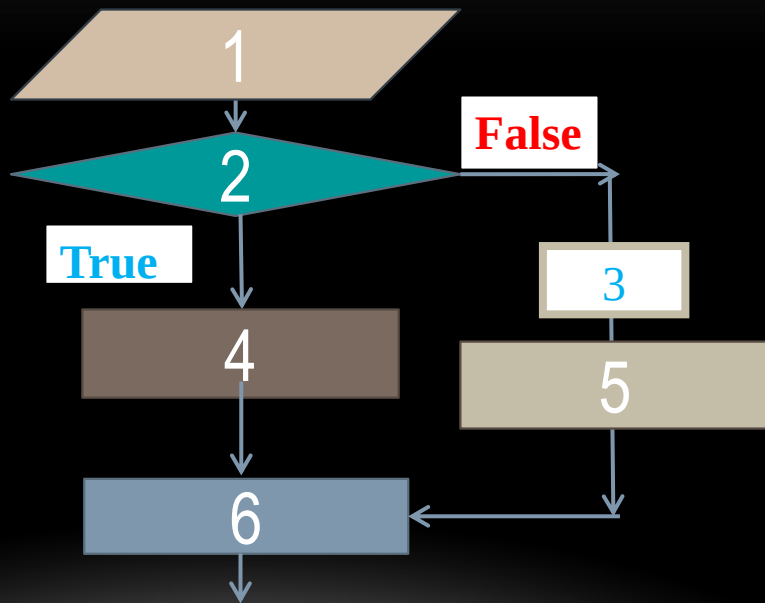


```
import random #導入隨機數庫
monsters = [
    {"名稱": "IT史萊姆", "生命": 20, "攻擊": 5, "閃避": 0.3},
    {"名稱": "IT哥布林", "生命": 50, "攻擊": 15, "閃避": 0.1},
    {"名稱": "IT巨魔", "生命": 100, "攻擊": 20, "閃避": 0.05}
]
monster = random.choice(monsters)
name="名稱"
print(f"出現了:{monster[name]}怪!!!")
```

- 請打開google classroom的“流程圖-if課堂練習”進行答題

課堂練習

請以流程圖及if條件式填寫遊戲的基
回合流程



WHILE TRUE: / WHILE 1:

設計一條程序讓同學可以選擇進入遊戲

while True:

Pchoice=int(input("你選擇你的行動選項:"))

if Pchoice == "1":

print("你選擇了戰鬥，準備開始！")

elif Pchoice == "2":

print("你選擇了防禦。")

elif Pchoice == "3":

print("你逃跑了，遊戲結束！")

break

print("退出遊戲!")

```
while True: #使用while True進入遊戲的循環中
    monster = random.choice(monsters) # 從列表/串列list中，隨機取出一個項目隨機選擇一個值"怪物"
    # 以下是玩家回合內容
    print("\n你的回合到了:")
    print("你的生命值:", player_life)
    print("怪物的生命值:", monster["生命"])
    print("1. 攻擊")
    print("2. 防禦")
    print("3. 逃跑")

    Pchoice = input("請選擇你的行動:")

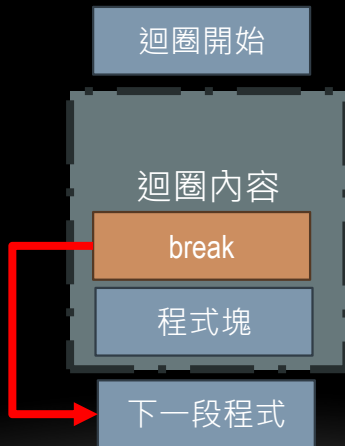
    if Pchoice == "1":
        # player選擇攻擊
        if random.random() > monster["閃避"]: # 使用random.random()產生0.1-1的隨機數
            damage = player_attack - (player_attack * monster["閃避"]) #傷害值的計算
            monster["生命"] -= damage
            print("你對怪物造成了", damage, "點傷害!")
        else:
            print("怪物閃避了你的攻擊!") #如果數值少於閃避值則完全閃避攻擊
    elif Pchoice == "2":
        # player選擇防禦
        player_defense += 0.1
        print("你進入防禦狀態，你的防禦力提升!")
    elif Pchoice == "3":
        print("你逃跑了，遊戲結束!")
        break
    else:
        print("請輸入有效的選項!")
        continue

    if monster["生命"] <= 0:
        print("你擊敗了IT怪物，獲得勝利!Y^_Y ")
        break
```

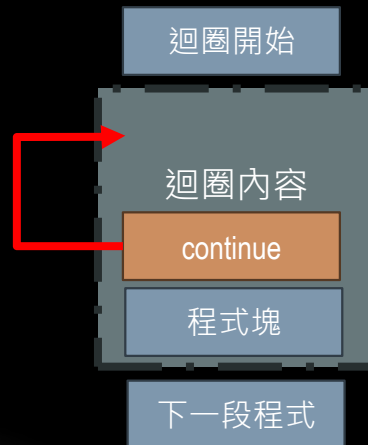
BREAK 和 CONTINUE

在迴圈中，可以使用「break 中斷」和「continue 跳出」兩個方法來「停止迴圈」，通常 break 和 continue 會搭配邏輯判斷一同使用。

break 會將整個迴圈停止



continue 是將迴圈目前執行的程式停止，然後再次執行迴圈



遊戲啟動2：勇者的戰鬥

- 角色屬性值的重要性
 - 攻擊力、防禦力和血量
- 小組討論：
 - 如何設定屬性才能提升遊戲的趣味和可玩性
 - 分享並摘錄一個包含主角和怪物的角色屬性



遊戲啟動2：勇者的戰鬥

- 遊戲戰鬥回合流程
- 使用迴圈和條件判斷來實現遊戲戰鬥的流程控制

```
Pchoice = input("請選擇你的行動：")

if Pchoice == "1":
    # player選擇攻擊
    if random.random() > monster["閃避"]: # 使用random.random()產生0.1-1的隨機數
        damage = player_attack - (player_attack * monster["閃避"]) # 傷害值的計算
        monster["生命"] -= damage
        print("你對怪物造成了", damage, "點傷害！")
    else:
        print("怪物閃避了你的攻擊！") # 如果數值少於閃避值則完全閃避攻擊
elif Pchoice == "2":
    # player選擇防禦
    player_defense += 0.1
    print("你進入防禦狀態，你的防禦力提升！")
elif Pchoice == "3":
    print("你逃跑了，遊戲結束！")
    break
else:
    print("請輸入有效的選項！")
    continue
```

遊戲啟動2：勇者的戰鬥

- 攻擊、防禦和傷害值的計算

```
if Pchoice == "1":  
    # player選擇攻擊  
    if random.random() > monster["閃避"]: # 使用random.random()產生0.1-1的隨機數  
        damage = player_attack - (player_attack * monster["閃避"]) # 傷害值的計算  
        monster["生命"] -= damage  
        print("你對怪物造成了", damage, "點傷害!")
```

- 處理生命值為0時的狀態

```
if monster["生命"] <= 0:  
    print("你擊敗了IT怪物，獲得勝利！Y^_^Y ")  
    break
```

```
if player_life <= 0:  
    print("你被怪物擊敗了，遊戲結束！X_X ")  
    print("="*30)  
    break
```

遊戲的數值及平衡性

我是遊戲修改大師

遊戲戰鬥事件和流程控制

- 遊戲中的角色強度、怪物難度、經驗和獎勵等方面的設計

```
if enemy["health"] <= 0:
    print("你打敗了", enemy["name"], "並獲得了", enemy["experience"], "點經驗值.")
    player["experience"] += enemy["experience"]
    enemies.remove(enemy)    # 在串列中刪除一個怪物
```

- 使用迴圈和條件判斷來控制遊戲戰鬥的事件(進階思考)
- 觸發必殺技攻擊或閃避效果等

```
if random.random() > monster["閃避"]:    # 使用random.random()產生0.1-1的隨機數
    damage = player_attack - (player_attack * monster["閃避"])    #傷害值的計算
    monster["生命"] -= damage
    print("你對怪物造成了", damage, "點傷害!")
else:
    print("怪物閃避了你的攻擊!")    #如果數值少於閃避值則完全閃避攻擊
elif Pchoice == "2":
    # player選擇防禦
    player_defense += 0.1
    print("你進入防禦狀態，你的防禦力提升!")
```

課題實作

- 主題實作：
- 請參考最基本的RPG遊戲程序，將程序改為具個人特色的遊戲

```
1 import random #導入隨機數庫
2
3 # 定義怪物"列表/串列list"(使用"字典dictionary"對應名稱及相關屬性)
4 monsters = [
5     {"名稱": "IT史萊姆", "生命": 20, "攻擊": 5, "閃避": 0.3},
6     {"名稱": "IT哥布林", "生命": 50, "攻擊": 15, "閃避": 0.1},
7     {"名稱": "IT巨魔", "生命": 100, "攻擊": 20, "閃避": 0.05}
8 ]
9
10 # 玩家初始屬性
11 player_life = 100 #生命值
12 player_attack = 20 #攻擊力
13 player_defense = 0.1 #防禦值
14 player_dodge = 0.15 #閃避值
15
16 # 遊戲開始
17 print("一隻可怕的怪物出現了!")
18 print("^^|_|^^")
19 print("!!--!! ")
20 print("  |  ")
21 print("你要與之戰鬥嗎?")
22
23 while True: #使用while True進入遊戲的循環中
24     monster = random.choice(monsters) # 從列表/串列list中，隨機取出一個項目隨機選擇一個值"怪物"
25     # 以下是玩家回合內容
26     print("\n你的回合到了:")
27     print("你的生命值:", player_life)
28     print("敵人:", monster["名稱"])
29     print("怪物的生命值:", monster["生命"])
30     print("1. 攻擊")
31     print("2. 防禦")
32     print("3. 逃跑")
33
34     Pchoice = input("請選擇你的行動:")
35
```

```
35
36 if Pchoice == "1":
37     # player選擇攻擊
38     if random.random() > monster["閃避"]: # 使用random.random()產生0.1-1的隨機數
39         damage = player_attack - (player_attack * monster["閃避"]) # 傷害值的計算
40         monster["生命"] -= damage
41         print("你對怪物造成了", damage, "點傷害!")
42     else:
43         print("怪物閃避了你的攻擊!") #如果數值少於閃避值則完全閃避攻擊
44
45 elif Pchoice == "2":
46     # player選擇防禦
47     player_defense += 0.1
48     print("你進入防禦狀態, 你的防禦力提升!")
49
50 elif Pchoice == "3":
51     print("你逃跑了, 遊戲結束!")
52     break
53
54 else:
55     print("請輸入有效的選項!")
56     continue
57
58 if monster["生命"] <= 0:
59     print("你擊敗了IT怪物, 獲得勝利! Y^Y ")
60     break
61
62 # 怪物回合
63 print("\n怪物回合:")
64 print("你的生命值:", player_life)
65 print("怪物的生命值:", monster["生命"])
66
67 if random.random() > player_dodge: # 使用random.random()產生0.1-1的隨機數
68     damage = monster["攻擊"] - (monster["攻擊"] * player_defense) #Player受傷害計算, 當player防禦力高於1時, 則不會再受傷害
69     player_life -= damage
70     print("怪物對你造成了", damage, "點傷害!")
71 else:
72     print("你閃避了怪物的攻擊!") #如果數值少於閃避值則完全閃避攻擊
73     print("=-*30)
74
75 if player_life <= 0:
76     print("你被怪物擊敗了, 遊戲結束! X_X ")
77     print("=-*30)
78     break
79
```

優秀作品分享及小結

這是我的RPG GAME